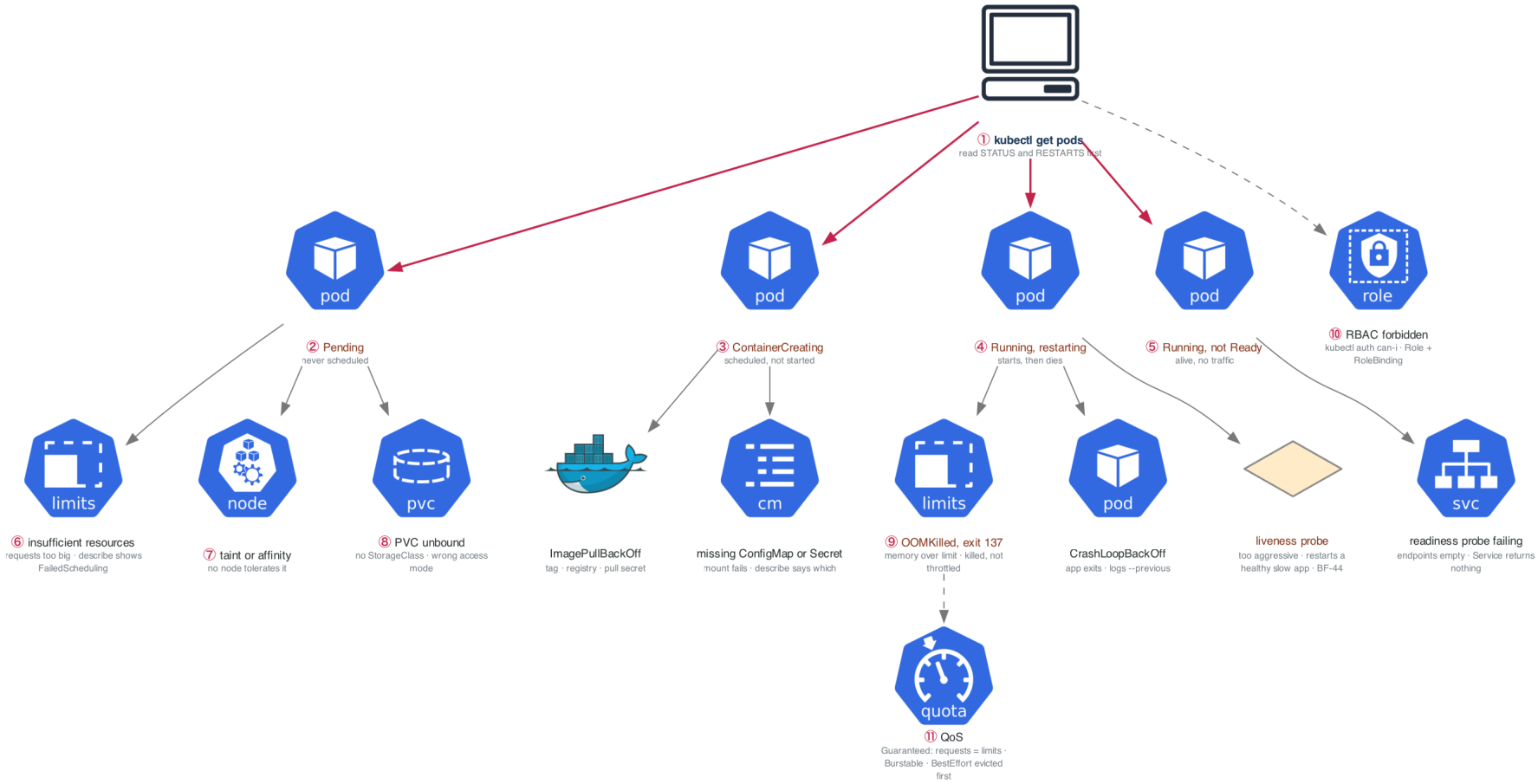
Nine ways a pod fails to start, and where each one lives, numbered

Week 15 · Figure 15.1 · figure with badges 1–11 · 2026-10-17



What to say, badge by badge

About three and a half minutes at a normal pace (383 words). Point at each badge as you say its line.

▶ **Open**
“Anyone can apply a manifest. What gets hired is diagnosing a broken cluster under time pressure while saying what you are thinking. This tree is the thing you say. The first word of STATUS picks the branch.”

1 **kubectl get pods**
“Two columns tell you which branch: STATUS and RESTARTS. Pending and ContainerCreating have zero restarts and never ran. Running with climbing restarts is dying after it starts. Running with zero Ready is alive but excluded.”

2 **Pending**
“The scheduler could not place it. `kubectl describe pod` shows a FailedScheduling event with the reason. There are three usual reasons, badges 6 to 8.”

3 **ContainerCreating**
“It has a node but the kubelet cannot start it. Almost always the image or a volume. Describe shows which, in the events at the bottom.”

4 **Running, restarting**
“The container starts and dies. The exit code and the previous container's logs tell you whether the app quit, the kernel killed it, or the probe did.”

5 **Running, not Ready**
“The process is alive but the readiness probe fails, so the Service removes it from the endpoints. The app looks down from outside and looks fine from inside.”

6 **insufficient resources**
“Requests larger than any node has free. On kind this is easy to trigger: ask for 100 CPUs. In production it is the usual Monday-morning cause.”

7 **taint or affinity**
“Every node has a taint the pod does not tolerate, or an affinity rule nothing satisfies. Describe names the taint.”

8 **PVC unbound**
“The claim is waiting for a volume that cannot be provisioned: no default StorageClass, or an access mode the class cannot give. Stateful pods stall here.”

9 **OOMKilled**
“Exit code 137. Memory over the limit is killed instantly, where CPU over the limit is only throttled. That asymmetry is the exit criterion and the question interviewers love. Requests equal to limits makes the pod Guaranteed and last to be evicted.”

10 **RBAC forbidden**
“Not a pod state, but it looks like one when a controller cannot create what it needs. `kubectl auth can-i --as` answers it in one line; a Role plus a RoleBinding fixes it.”

11 **QoS**
“Guaranteed when every container's requests equal its limits. Burstable when requests are lower. BestEffort when there are none, and those die first under pressure.”

■ **Close**
“Read the status, pick the branch, run the one command that proves the leaf, say it out loud. Friday you build five of these on demand; they are Gate 4, two weeks from now.”